

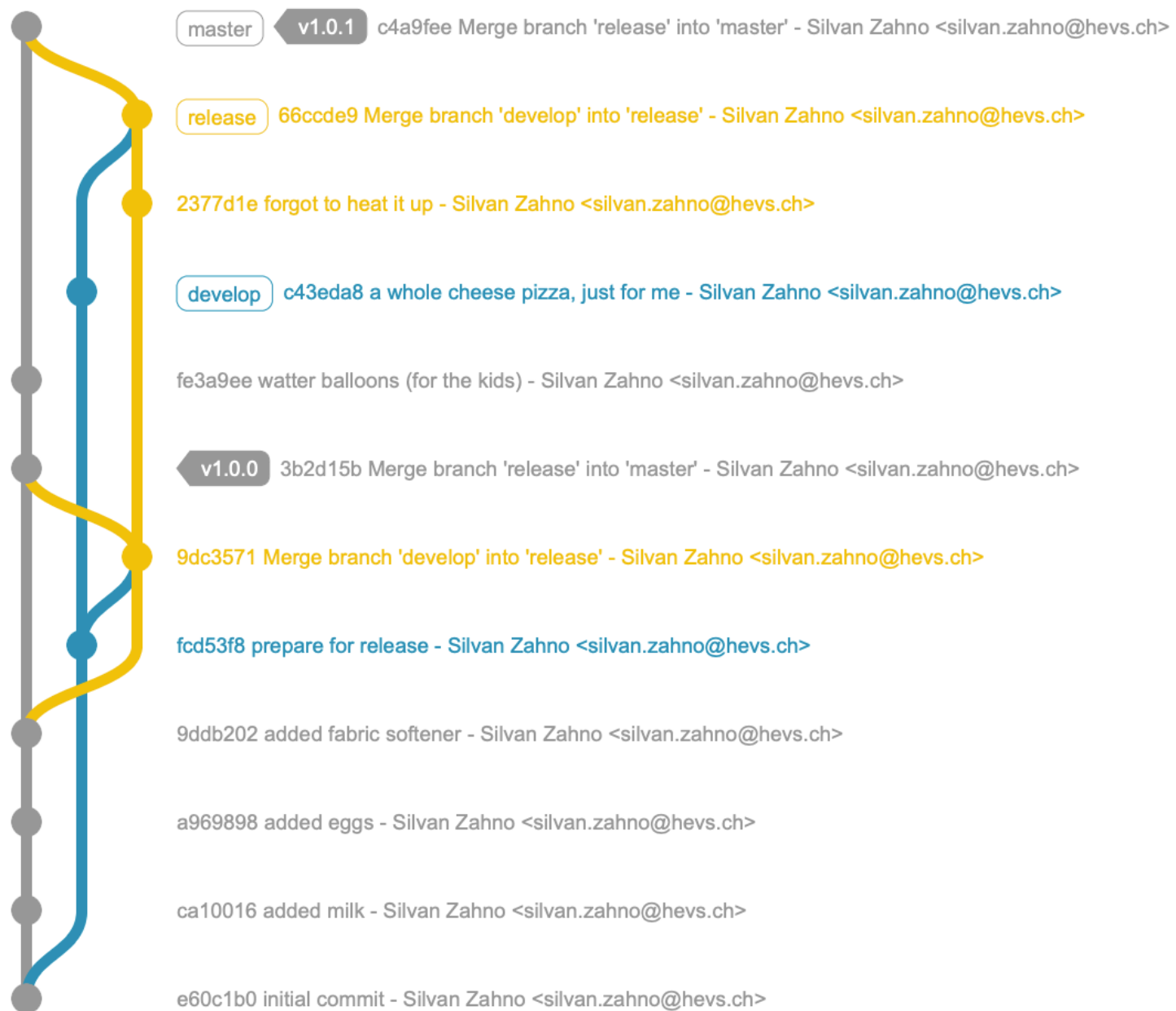
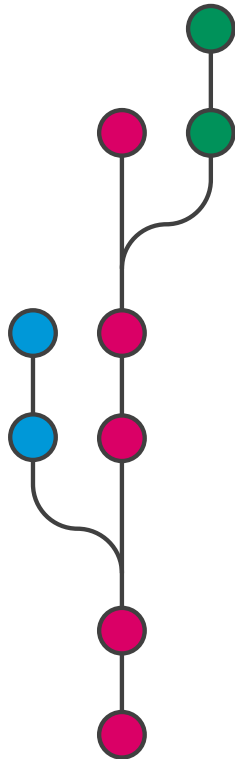
Votre système actuel



```
.  
├── fichier important copy.txt  
├── fichier important v1.txt  
├── fichier important v2.1 backup.txt  
├── fichier important v2.1 backup.txt.old  
├── fichier important v2.1.txt  
├── fichier important v2.txt  
└── fichier important.txt
```

1 répertoire, 7 fichiers

ZaS



Outils possibles

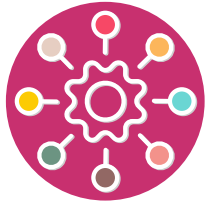
Git (git)



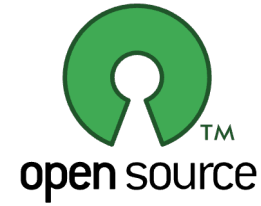
Subversion (svn)



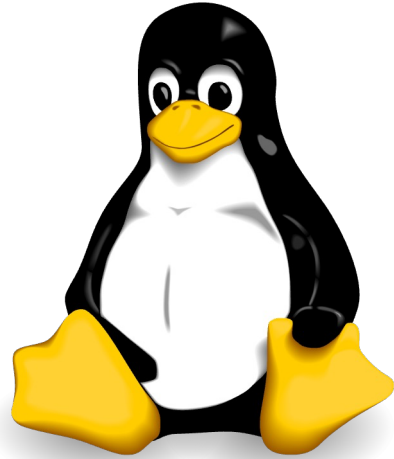
Mercurial (hg)



Linux Torvalds



Linux (1991)

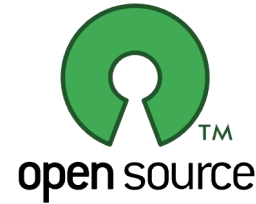
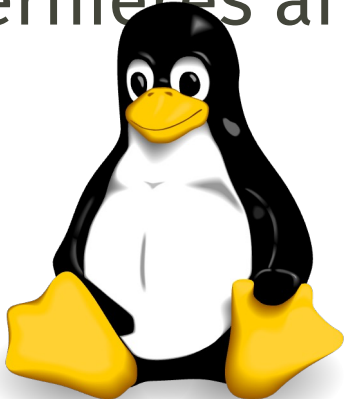


Git (2005)



Pourquoi Linux a besoin de git

Linux est devenu le plus grand projet de développement collaboratif de l'histoire de l'informatique au cours des 30 dernières années.



- 600 distributions Linux actives
- 85% de tous les smartphones
- 500 superordinateurs de pointe
- > 27,8 millions de lignes de code
- > 12 000 contributeurs
- > 1 million de commits

<https://truelist.co/blog/linux-statistics/>

Pourquoi pour un Power&Control ou Design&Material git est nécessaire

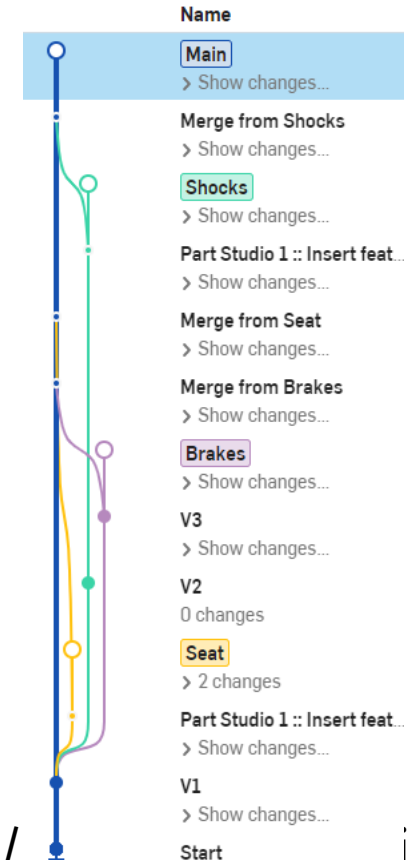


Version

- Suivi des modifications
- Collaboration
- Capacités de retour en arrière
- Documentation
- Déploiement



AUTODESK
Vault



<https://truelist.co/>

Plateformes Git



Gitlab

<https://gitlab.com>

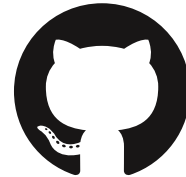
<https://gitlab.hevs.ch>



GitLab

Github

<https://github.com>



GitHub

Bitbucket

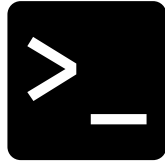
<https://bitbucket.com>



Bitbucket

Outils Git

Ligne de commande



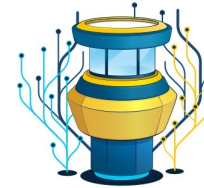
Fork



Sublime Merge



Tower



Git Cola



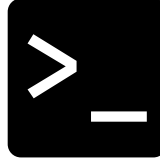
SmartGit



Git Kraken



Git ligne de commande



git status

git diff

git add <file>

git diff --staged

git reset <file>

git commit -m "<commit message>"

git fetch <remote>

git merge <remote> <branch>

git push <remote> <branch>

git pull

```
zas@zas@zac ~ - zsh - 99x52
Last login: Tue Mar  8 09:26:26 on ttys004
(zas@zac)~(base)
$ git --help
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
      [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
      [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
      [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
      [--super-prefix=<path>] [--config-env=<name>=<envvar>]
      <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status

grow, mark and tweak your common history
  branch     List, create, or delete branches
  commit     Record changes to the repository
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Reset current HEAD to the specified state
  switch     Switch branches
  tag        Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

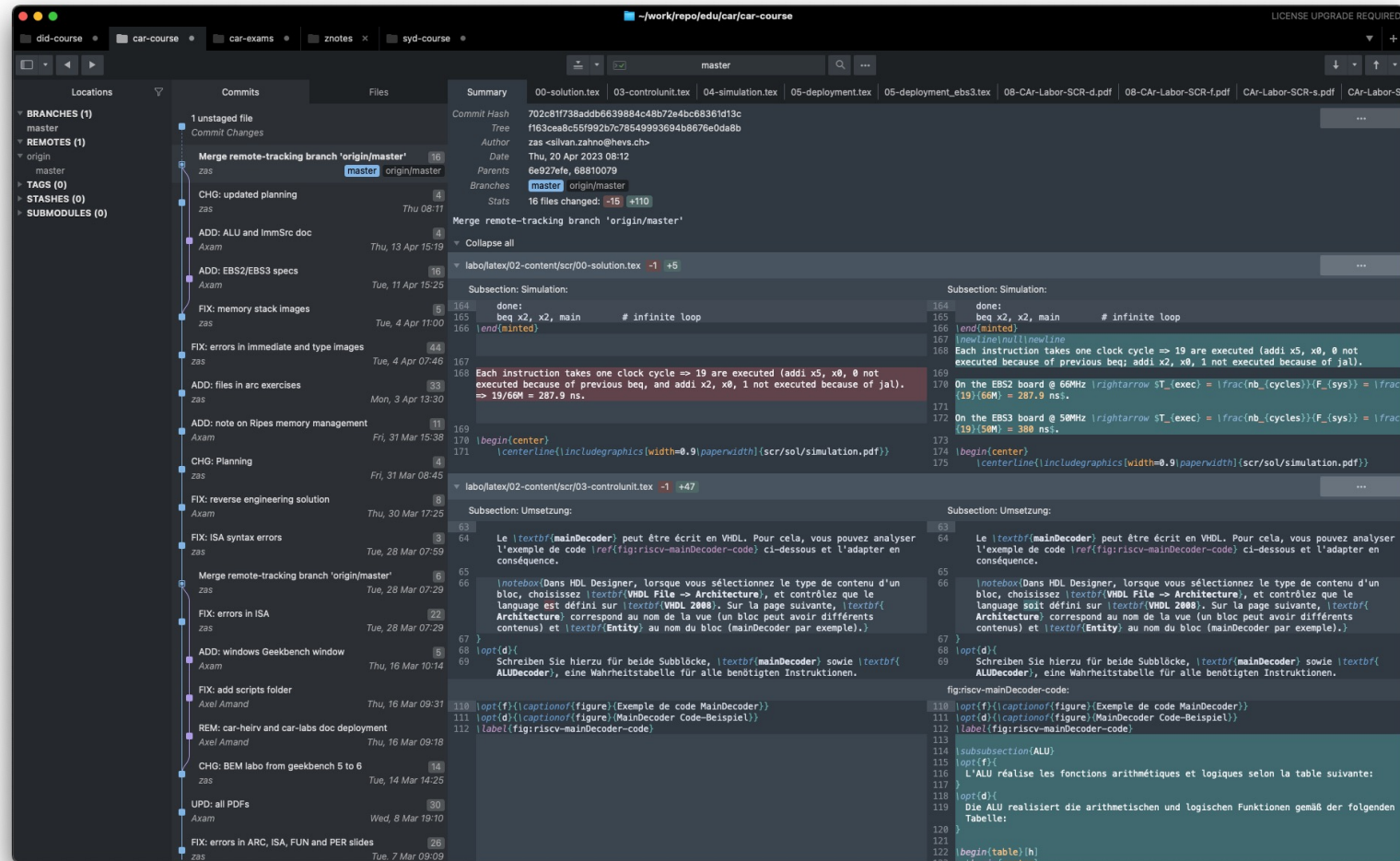
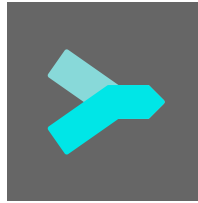
(zas@zac)~(base)
$
```

git commands

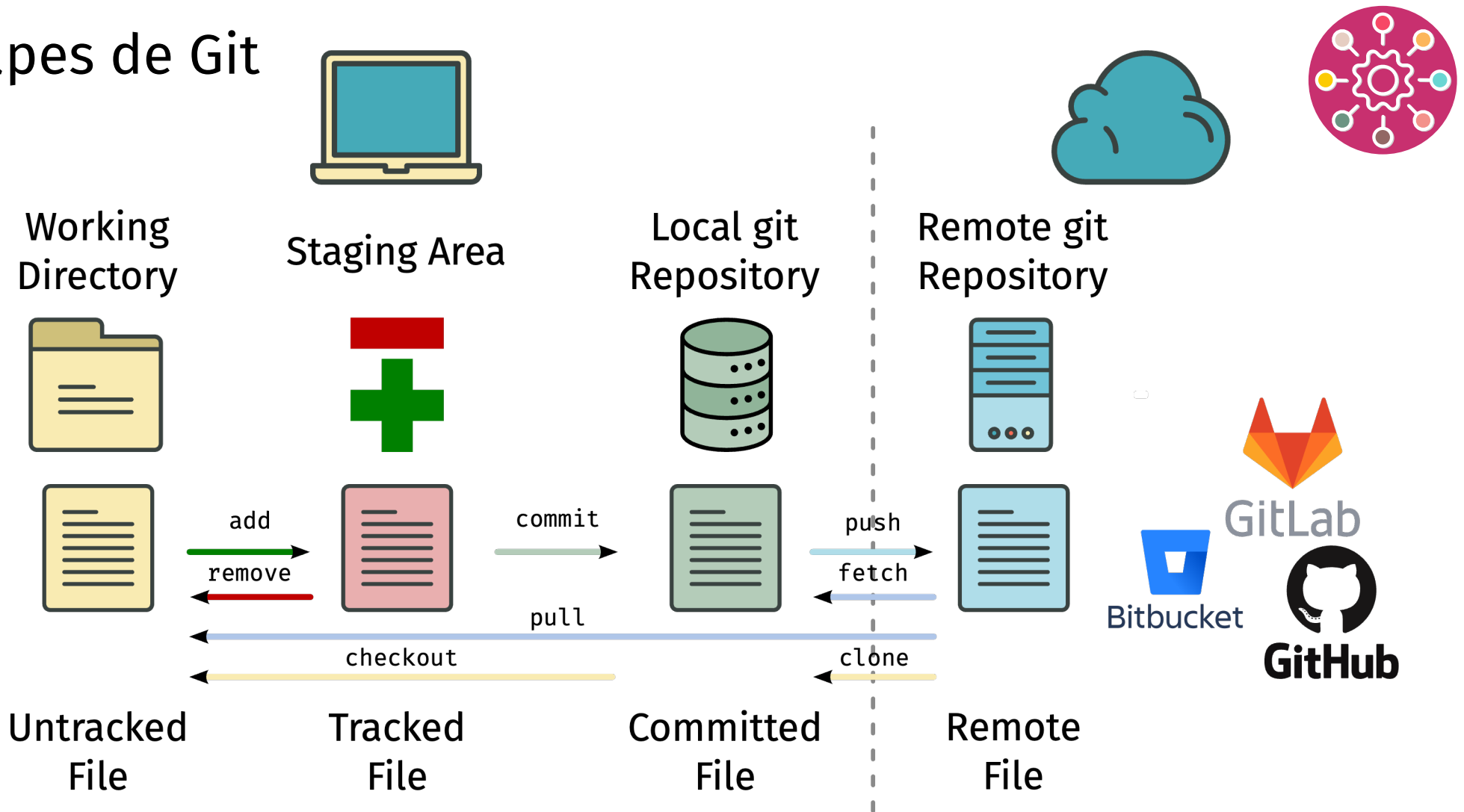


Command	Description	Command	Description
Démarrer une zone de travail		Développez, marquez et modifiez votre histoire commune	
clone	Cloner un dépôt dans un nouveau répertoire	branch	Lister, créer ou supprimer des branches
init	Créer un dépôt Git vide ou réinitialiser un dépôt existant	checkout	Changer de branche ou restaurer les fichiers de l'arborescence de travail
Travailler sur la modification en cours		commit	Enregistrer les modifications apportées à la base de données
add	Ajouter le contenu d'un fichier à l'index	diff	Afficher les changements entre les livraisons, les livraisons et l'arbre de travail, etc.
mv	Déplacer ou renommer un fichier, un répertoire ou un lien symbolique	merge	Joindre deux ou plusieurs historiques de développement
reset	Réinitialiser le HEAD actuel à l'état spécifié	rebase	Réappliquer les commits par-dessus un autre conseil de base
rm	Supprimer des fichiers de l'arbre de travail et de l'index	tag	Créer, lister, supprimer ou vérifier un objet d'étiquette
Examiner l'historique et l'état		Collaborer	
log	Afficher les journaux de livraison	fetch	Télécharger des objets et des références à partir d'un autre dépôt
show	Afficher différents types d'objets	pull	Récupérer et intégrer des objets d'une autre base de données ou d'une branche locale
status	Afficher l'état de l'arbre de travail	push	Mettre à jour les références distantes ainsi que les objets associés

Sublime Merge

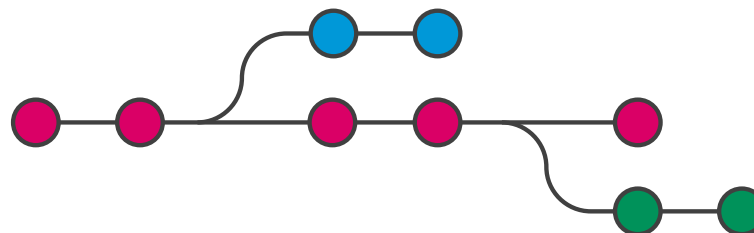


Étapes de Git





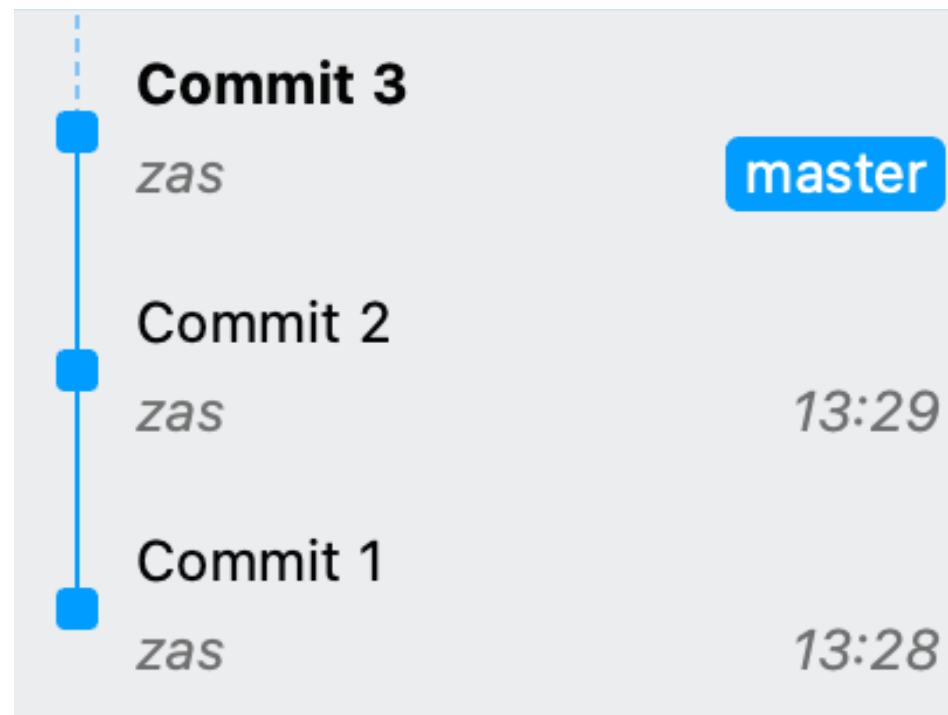
Git Branch et Merge Exemple



Branch et Merge

Initial repo state

Chaque repo a une branche **main** ou une branche **master** comme branche par défaut.



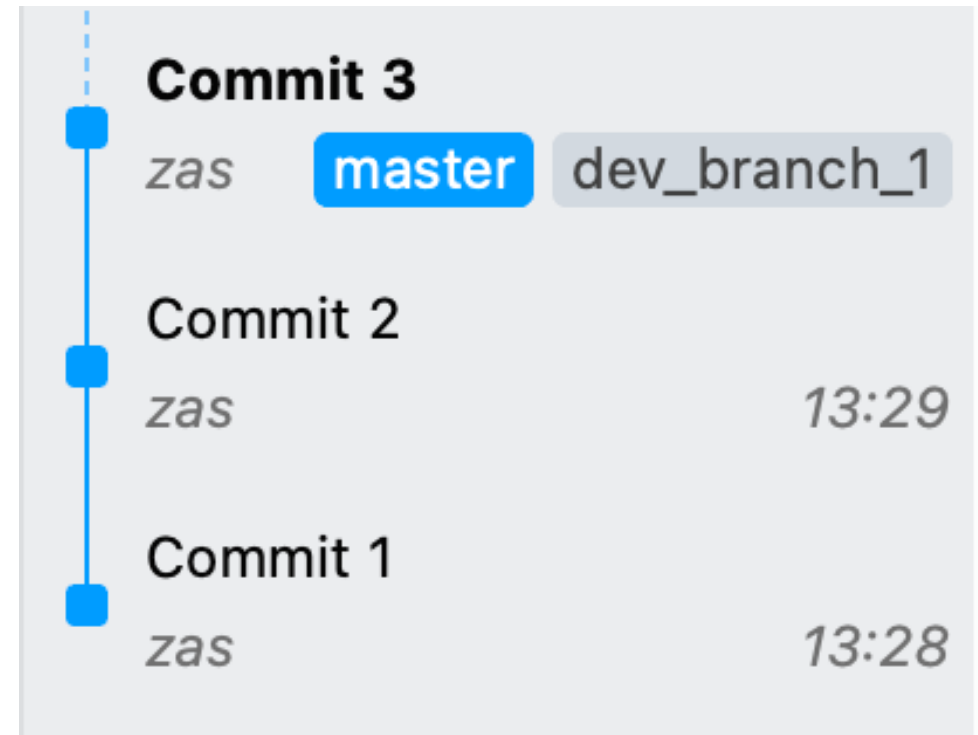
Branch et Merge

Create branch dev_branch_1



Create branch dev_branch_1

```
$ git branch dev_branch_1
```



Branch et Merge

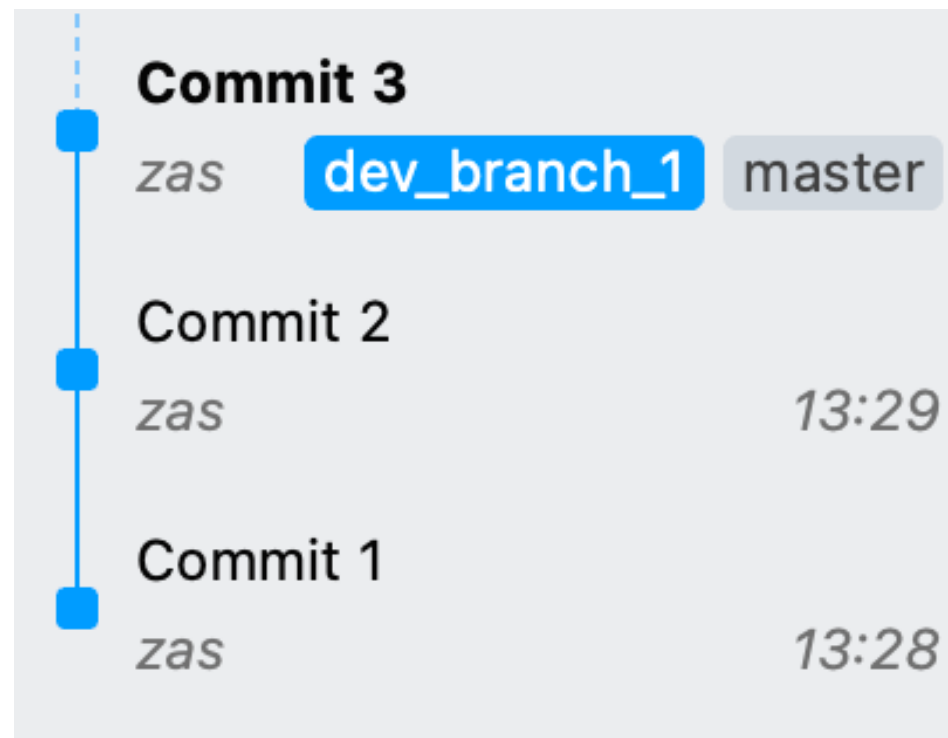
Checkout branch dev_branch_1

Vérifier sur quelle branche nous nous trouvons

```
$ git branch
```

Checkout branch dev_branch_1

```
$ git checkout dev_branch_1
```



Branch et Merge

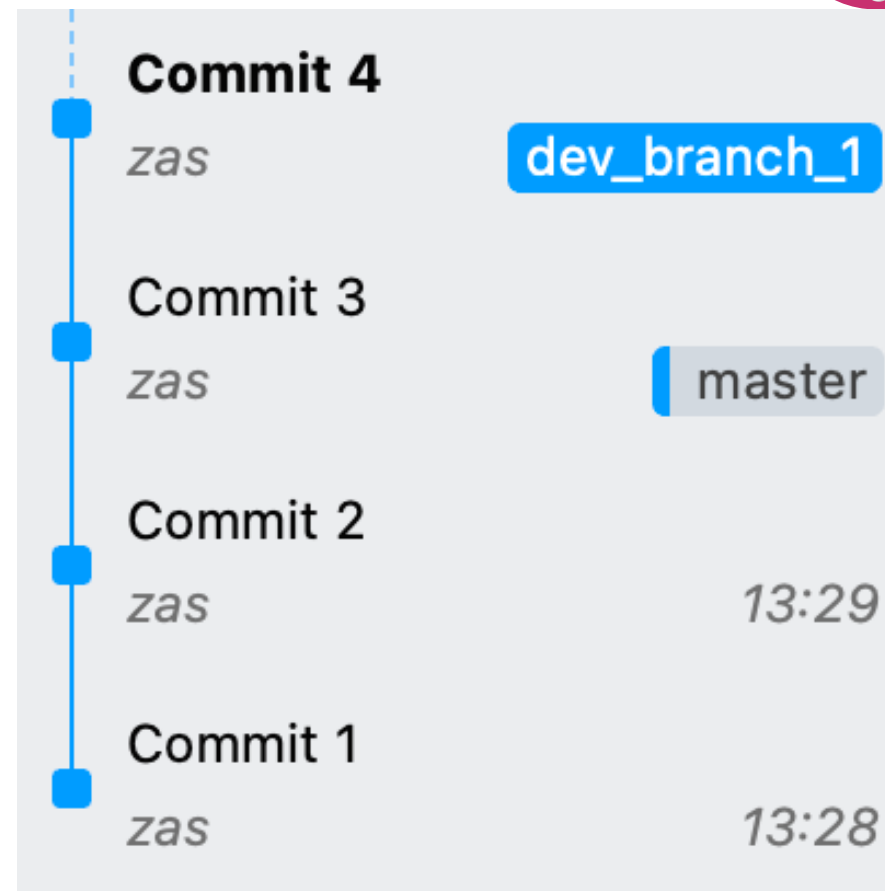
Commit on dev_branch_1

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 4"
```



Branch et Merge

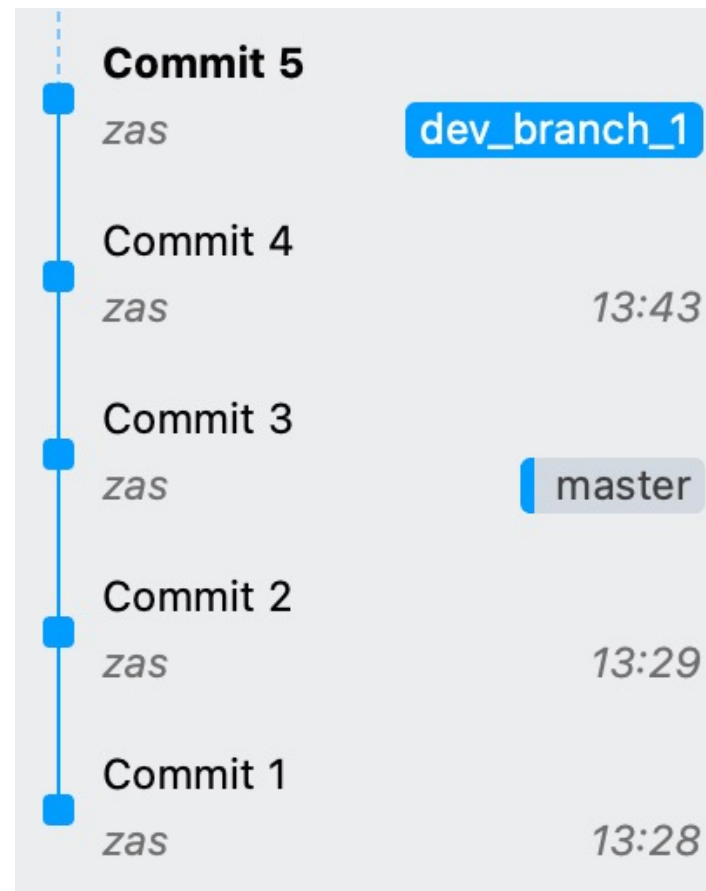
Commit on dev_branch_1

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 5"
```



Branch et Merge

Commit on master branch

Checkout master branch

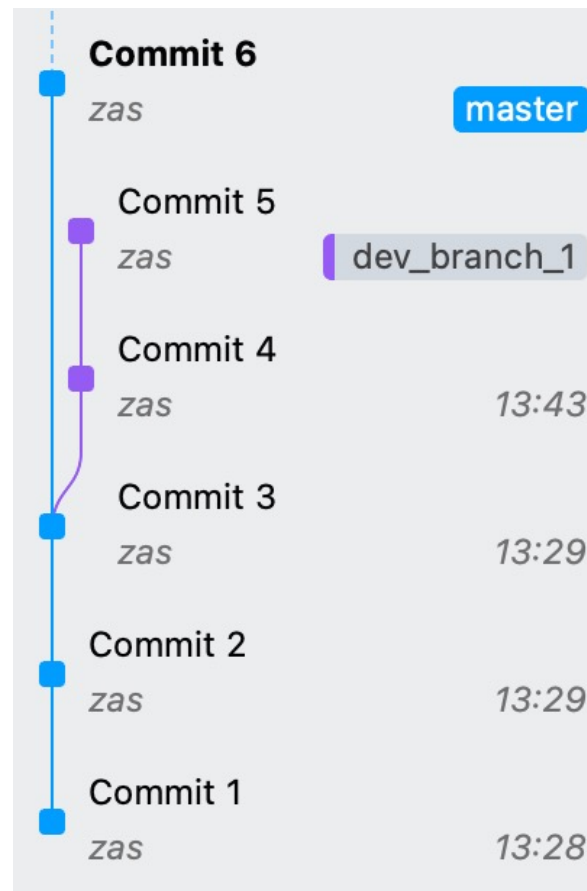
```
$ git checkout master
```

Stage new file

```
$ git add file.md
```

Commit stages files

```
$ git commit -m "Commit 6"
```



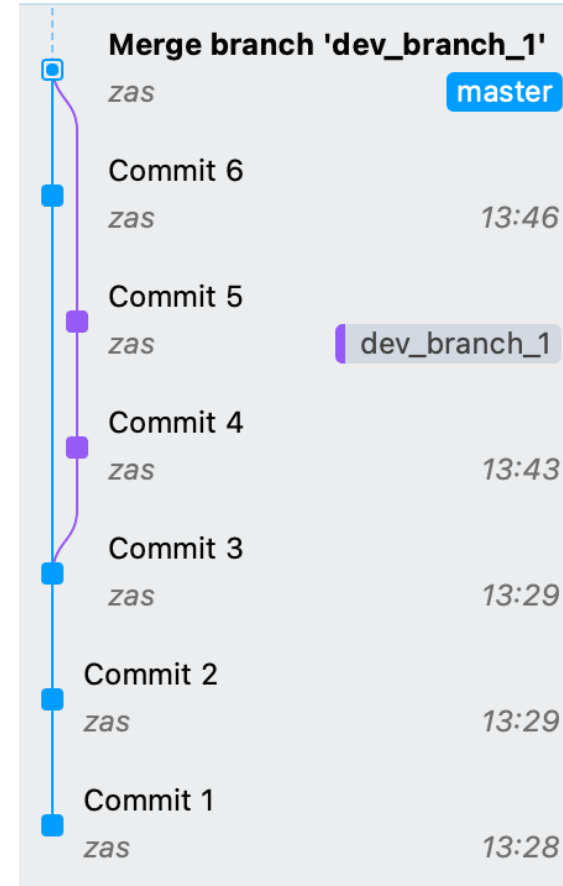
Branch et Merge

Three way merge master and dev_branch_1



Merge dev_branch_1 into master

```
$ git merge dev_branch_1
```



Demo



Locations

▼

▼ BRANCHES (1)
master
▼ REMOTES (1)
origin
master
▼ TAGS (0)
▼ STASHES (0)
▼ SUBMODULES (0)

Commits

▼

1 unstaged file
Commit Changes
Merge remote-tracking branch 'origin/master'
CHG: updated planning
ADD: ALU and ImmSrc doc
ADD: EBS2/EBS3 specs
FIX: memory stack images
FIX: errors in immediate and type images
ADD: files in arc exercises
ADD: note on Ripes memory management
CHG: Planning
FIX: reverse engineering solution
FIX: ISA syntax errors
Merge remote-tracking branch 'origin/master'
FIX: errors in ISA
ADD: windows Geekbench window
FIX: add scripts folder
REM: car-heiv and car-labs doc deployment
CHG: BEM labo from geekbench 5 to 6
UPD: all PDFs
FIX: errors in ARC, ISA, FUN and PER slides

Files

▼

00-solution.tex
03-controlunit.tex
04-simulation.tex
05-deployment.tex
05-deployment_ebs3.tex
08-CAR-Labor-SCR-d.pdf
08-CAR-Labor-SCR-f.pdf
CAR-Labor-SCR-s.pdf
CAR-Labor-SCR-t.pdf

Summary

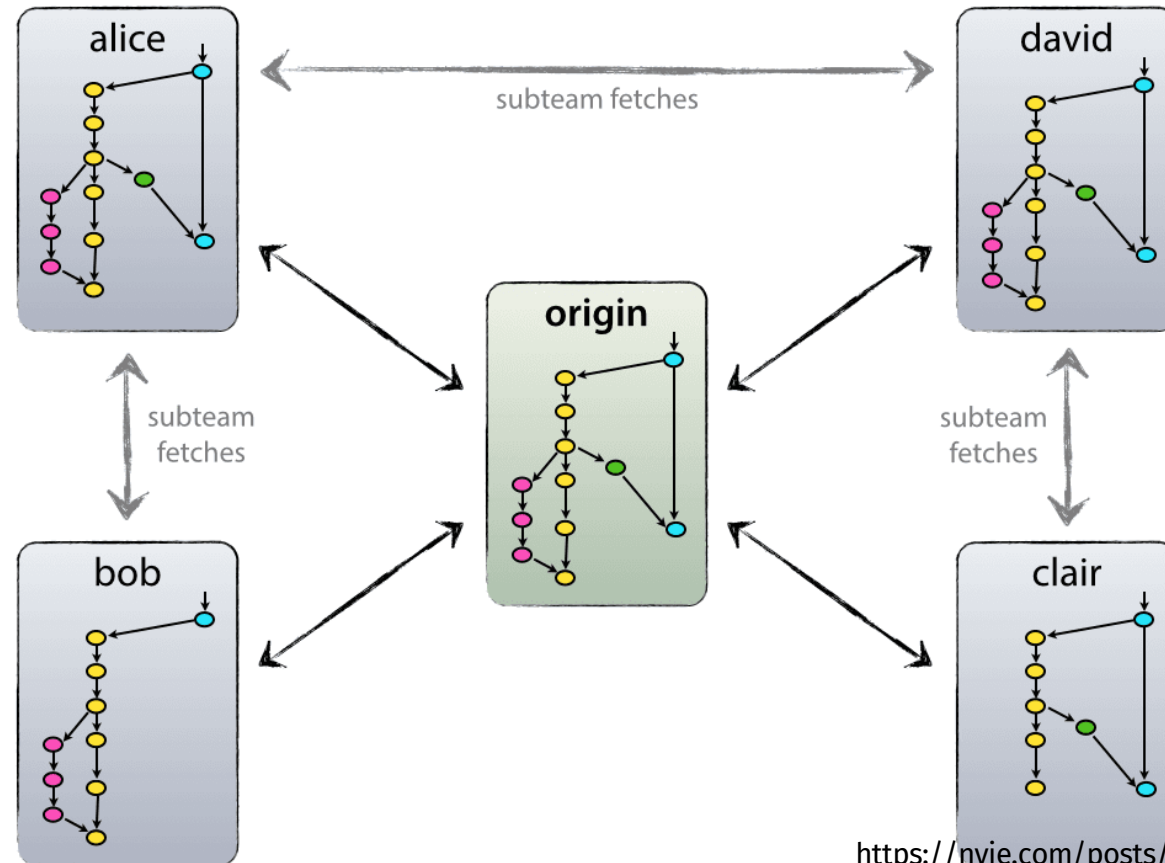
▼

Commit Hash
Tree
Author
Date
Parents
Branches
Stats
Merge remote-tracking branch 'origin/master'
Collapse all
Subsection: Simulation:
done:
beq x2, x2, main # infinite loop
end(minted)
Each instruction takes one clock cycle => 19 are executed (addi x5, x0, 0 not executed because of previous beq; addi x2, x0, 1 not executed because of jal).
On the EBS2 board @ 66MHz $\rightarrow 19$ are executed (addi x5, x0, 0 not executed because of previous beq; addi x2, x0, 1 not executed because of jal).
On the EBS3 board @ 50MHz $\rightarrow 19$ are executed (addi x5, x0, 0 not executed because of previous beq; addi x2, x0, 1 not executed because of jal).
Subsection: Umsetzung:
Le `\textbf{mainDecoder}` peut être écrit en VHDL. Pour cela, vous pouvez analyser l'exemple de code `\ref{fig:riscv-mainDecoder-code}` ci-dessous et l'adapter en conséquence.
Dans HDL Designer, lorsque vous sélectionnez le type de contenu d'un bloc, choisissez `\textbf{VHDL File}` -> `Architecture`, et contrôlez que le langage est défini sur `\textbf{VHDL 2008}`. Sur la page suivante, `\textbf{Architecture}` correspond au nom de la vue (un bloc peut avoir différents contenus) et `\textbf{Entity}` au nom du bloc (`mainDecoder` par exemple).
Schreiben Sie hierzu für beide Subblöcke, `\textbf{mainDecoder}` sowie `\textbf{ALUDecoder}`, eine Wahrheitstabelle für alle benötigten Instruktionen.
figriscv-mainDecoder-code:
Example de code MainDecoder)
MainDecoder Code-Beispiel)
figriscv-mainDecoder-code)
Subsubsection: ALU
L'ALU réalise les fonctions arithmétiques et logiques selon la table suivante:
Die ALU realisiert die arithmetischen und logischen Funktionen gemäß der folgenden Tabelle:
begin(table)[h
end(table)



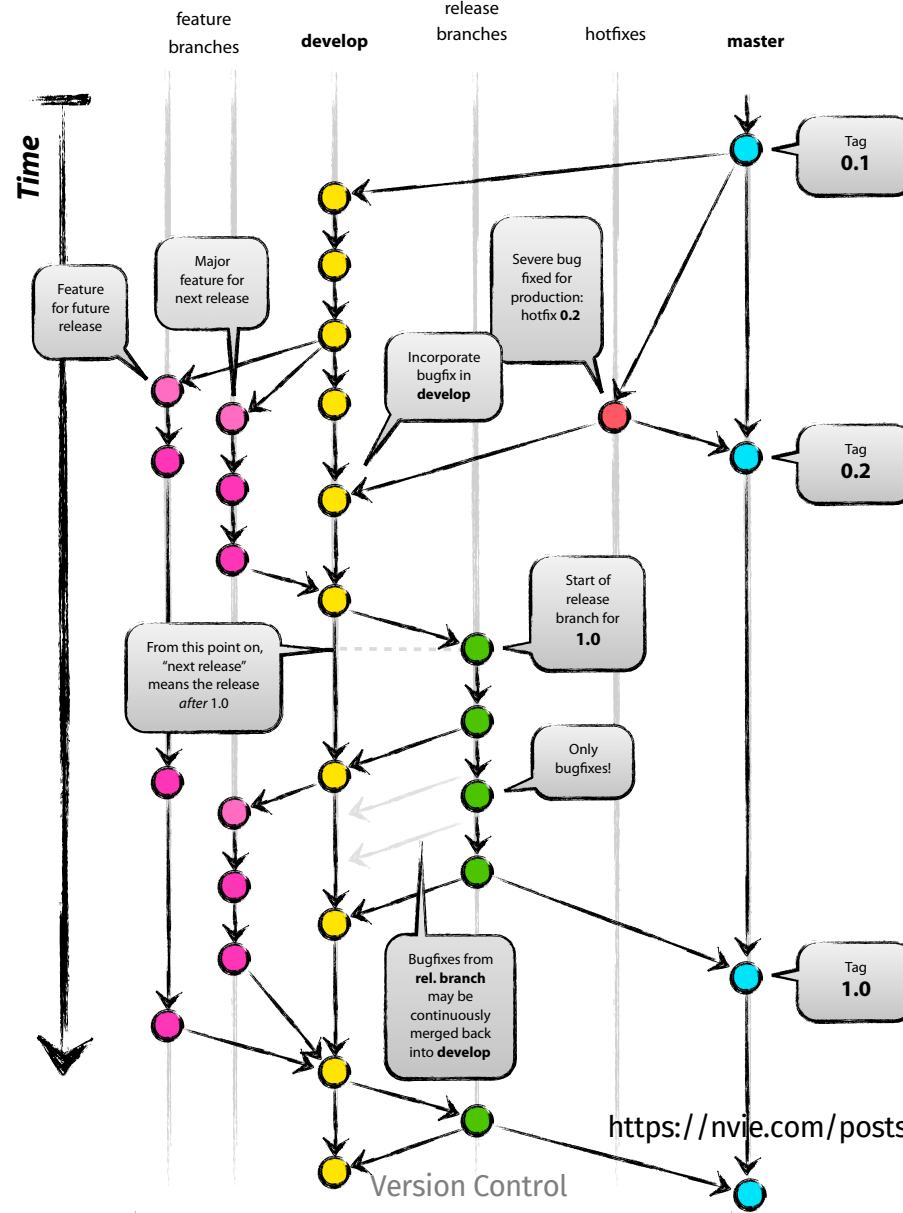
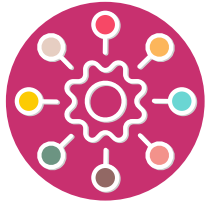
Gitflow

Gitflow Collaboration

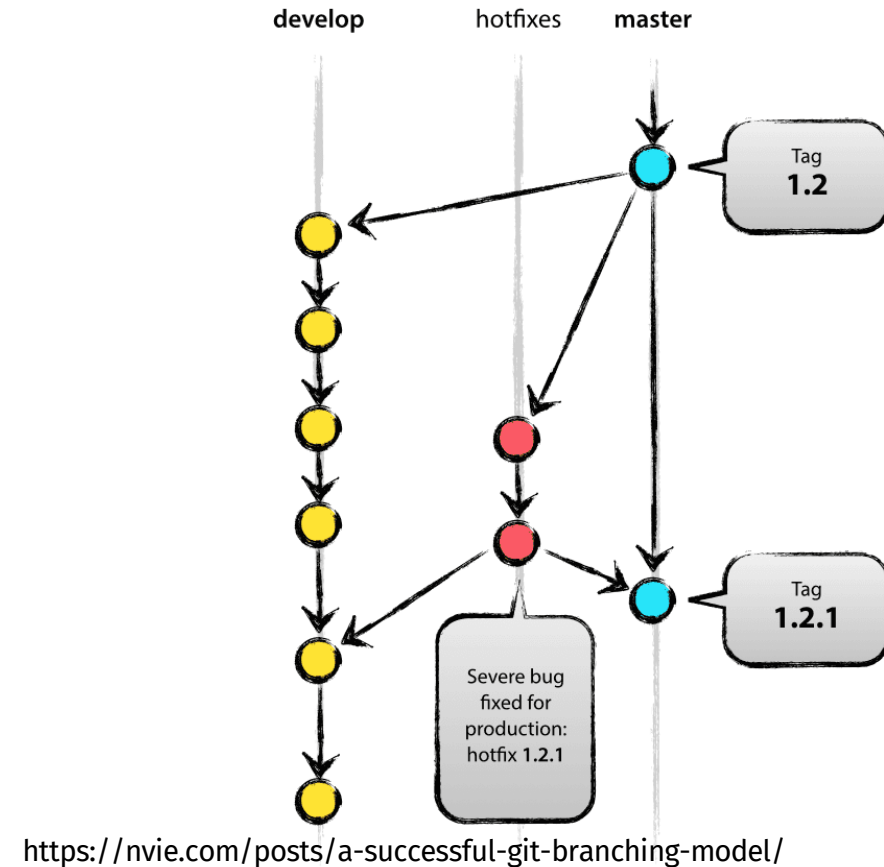
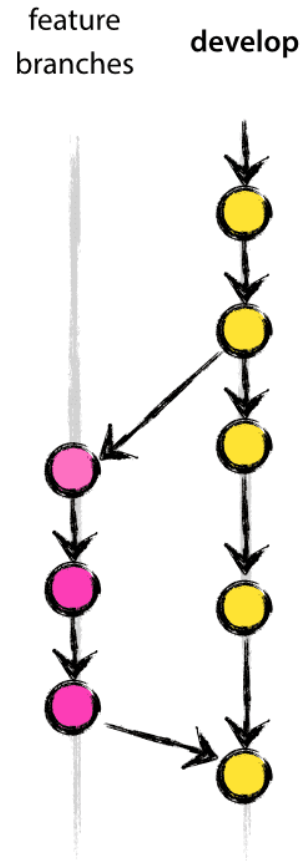
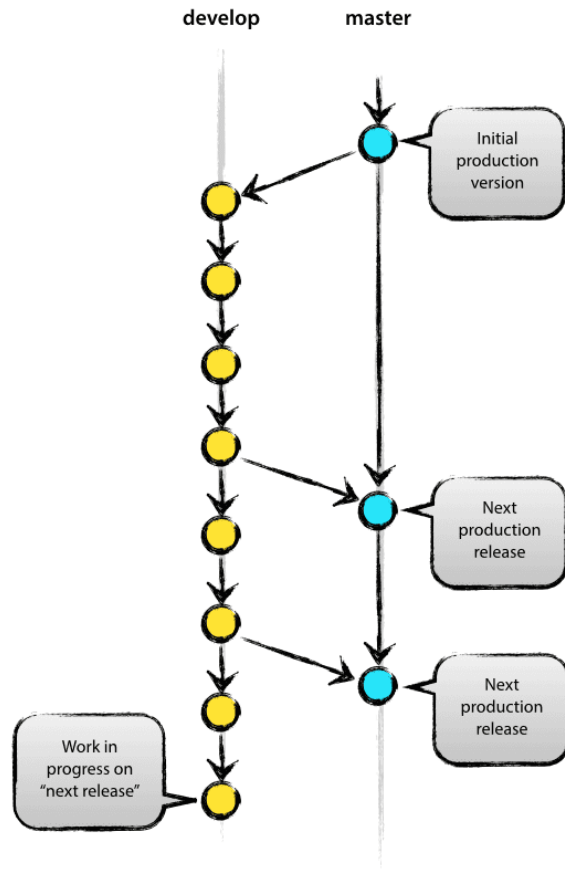


<https://nvie.com/posts/a-successful-git-branching-model/>

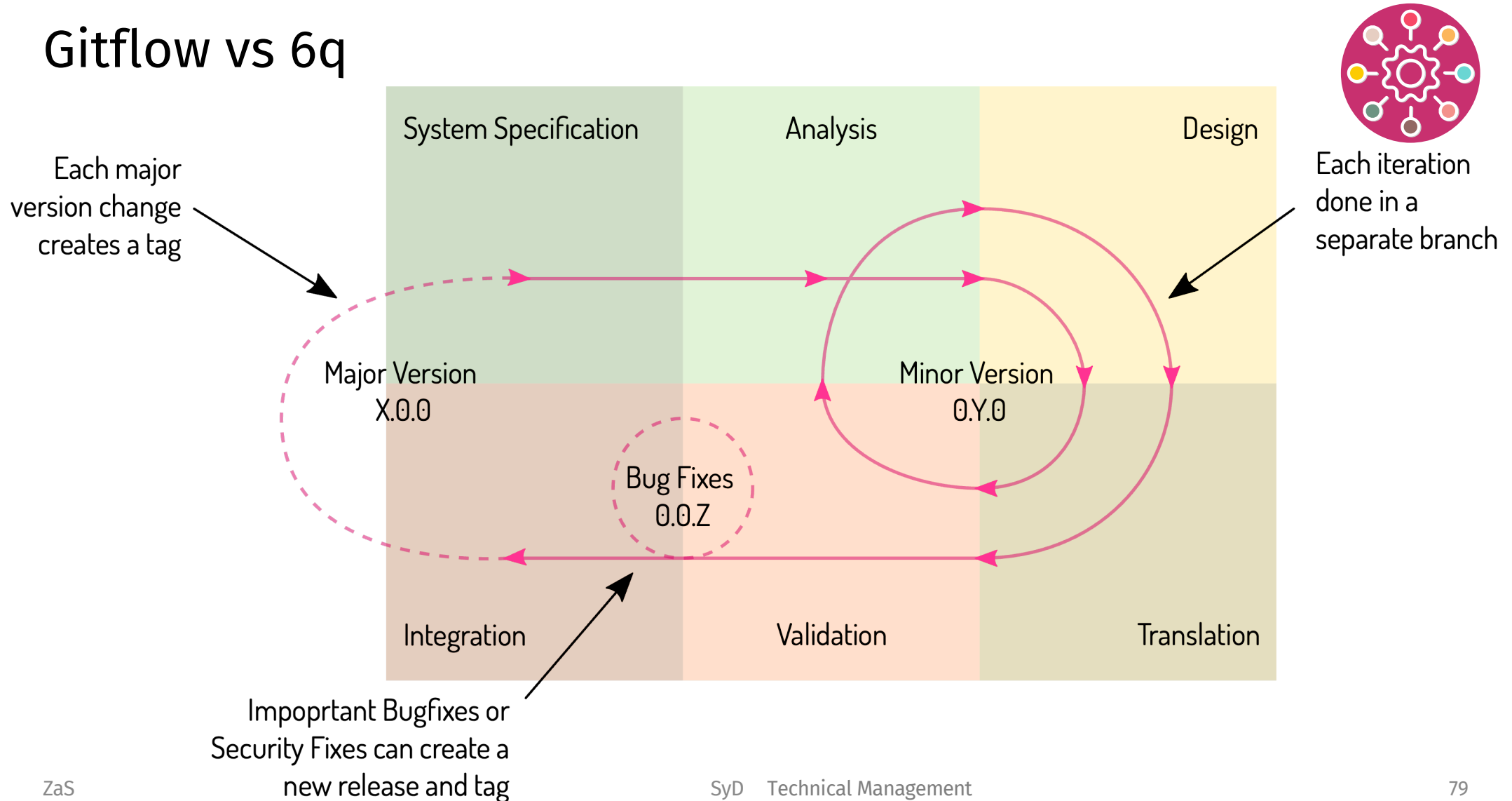
Gitflow



Gitflow Branching



Gitflow vs 6q





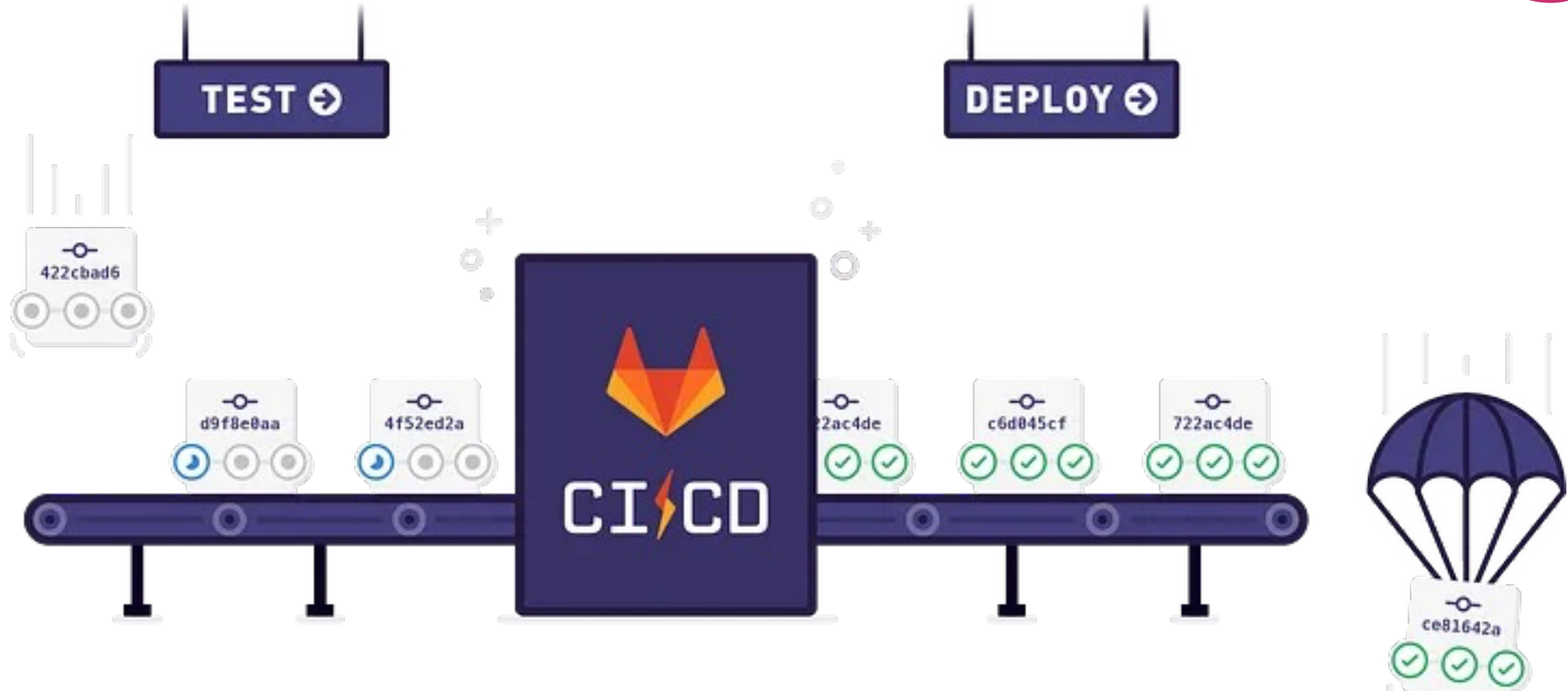
Git CI/CD

Qu'est-ce que la CI/CD ?



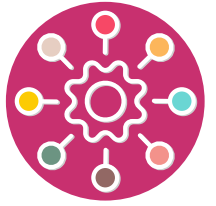
- L'intégration continue (Continuous Integration, CI) est la pratique qui consiste à intégrer fréquemment des modifications de code dans un référentiel partagé, qui est ensuite automatiquement construit et testé.
- La livraison continue (Continuous Delivery, CD) va encore plus loin en déployant automatiquement les modifications du code dans des environnements de type production afin de les tester et de les valider.
- L'automatisation des tests est un élément essentiel de la CI/CD, car elle permet de détecter les bogues et autres problèmes à un stade précoce du processus de développement.
- Les outils les plus courants sont GitLab CI/CD, Github Actions, Jenkins, CircleCI et Travis CI.

Qu'est-ce que la CI/CD ?

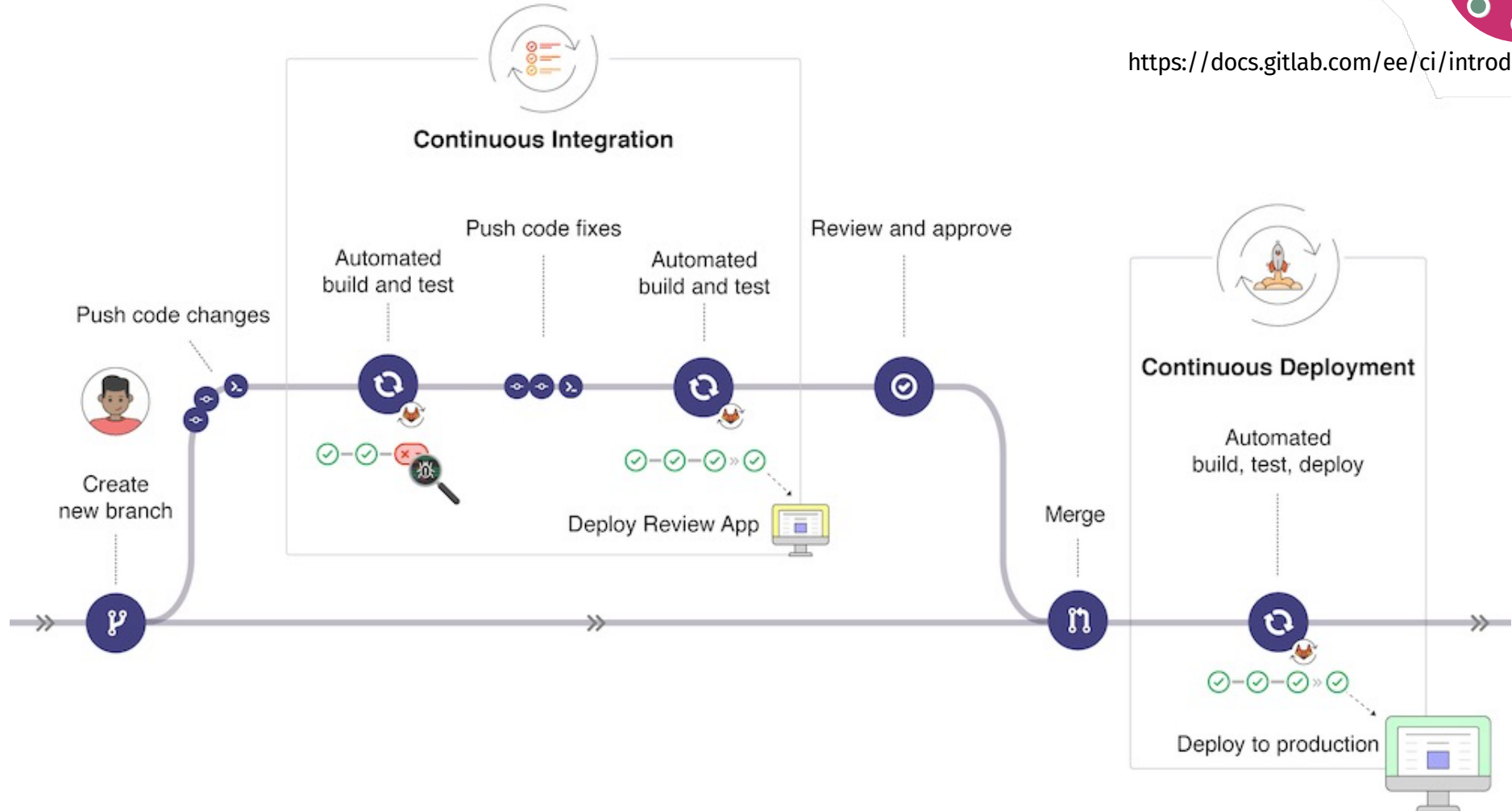


<https://about.gitlab.com/features/continuous-integration/>

Gitlab Workflow



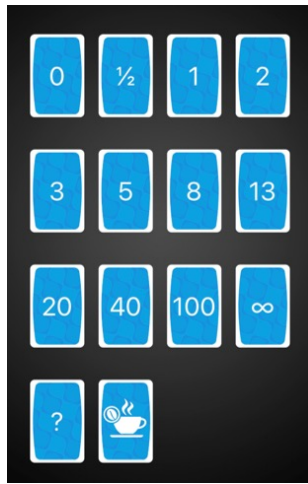
<https://docs.gitlab.com/ee/ci/introduction/>



Estimation des tâches

Estimation 4

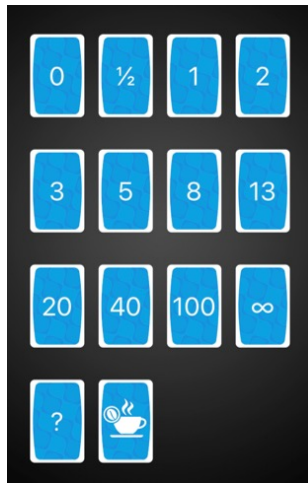
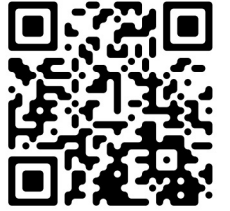
Créer un Star Wars
Star Destroyer (4784pcs)
X Point(s)



Estimation des tâches

Estimation 5

Créer un Star Wars
Star Destroyer (37pcs)
X Point(s)





SYSTEMS DESIGN / INTRODUCTION TO GIT - PART A



Introduction to git - Part A

Installation & Setup



Contents

1 Goal	1
2 Installation	2
3 Markdown	5
4 Outro	7
A GIT commands	8
B Most used Git commands	9

SYSTEMS DESIGN / INTRODUCTION TO GIT - PART B



Introduction to git - Part B



Contents

1 Goals	2
2 Outils	2
3 Basis Operationen	3
4 Branch and Merge	11
5 Gitgraph	15
6 Gitflow	16
7 Extras	18